

Edge weight

0-1 2

1-2 3

0-3 6

1-4 5

Minimum cost: 16

Execution time: 0.0000

Floyd's Algorithm

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define INF 99
```

```
void floyd (int A[10][10], int n) {
```

```
    int i, j, k;
```

```
    for (k=0; k<n; k++) {
```

```
        for (i=0; i<n; i++) {
```

```
            for (j=0; j<n; j++) {
```

```
                if (A[i][k] + A[k][j] < A[i][j]) {
```

```
                    A[i][j] = A[i][k] + A[k][j];
```

```
                }
```

```
            }
```

```
        }
```

```
    printf("\n Shortest distance matrix: ");
```

```
    for (i=0; i<n; i++) {
```

```
        for (j=0; j<n; j++) {
```

```
            if (A[i][j] == INF)
```

```
                printf("INF ");
```

```
            else
```

```
                printf("%d ", A[i][j]);
```

```
        }
```

```
    printf("\n");
```

2

3

```
int main() {
    int i, j, k, n;
    int cost[10][10], A[10][10];
    printf("Enter no. of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix: \n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            A[i][j] = cost[i][j];
        }
    }
}
```

4

clock = start, end;

start = clock();

floyd(A, n);

end = clock();

double time = (double)(end - start) /
CLOCKS_PER_SEC;

printf("Execution time: %.f", time);

return 0;

Output

Enter the number of vertices: 3

Enter the cost matrix:

0 4 99 5 99

99 0 1 99 6

2 99 0 3 99

99 99 1 0 2

1 99 99 4 0

Shortest distance Matrix:

0 4 5 5 7

3 0 1 4 6

2 6 0 3 5

3 7 1 0 2

1 5 5 4 0

Execution time: 0.0000